

Jacob Barnhill, Marquise Douglas,
Malloy Hawkins, Karma McCoy, Michael Poe
CSC 4033 - 001
Group 3
10/XX/2026

Product Backlog

Travel Aggregate: to be used by people trying to plan a trip, it takes in all info of how to get from point a to point b with public transit.

Order of Features:

1. Build web scraper(s)
2. Aggregate all available information on public (possibly private) transportation
3. Create a specific algorithm that will calculate the optimal routes
4. Build UI

What We Have:

- Transit API (Free)
- SerpAPI (Google flights API Scraper (Free?))
- Greyhound Bustracker API Github
- Uber API (Free)
- Amtrak Scraper (Need to build)

Missing Features:

- Calculate total trip time
- Calculate trip via multitude of transportation
- Filter for lowest cost
- Filter for lowest time

Research – Checkpoint 2

Take the items at the top of your backlog and find out how they would be built, before you write stories from them. A team can break work into tasks only if it knows roughly how the work will be done. Cohn says breaking a story into tasks “can only be done with at least a minimal design in mind.” This part is where your team builds that knowledge, as your learning plan did for the skeleton.

1. **Choose your items.** Take the items at the top of your backlog, as many as you can research well today.
 - a.
2. **List what you do not know.** For each item, list every thing you would have to do that your team does not yet know how to do. Include what you marked in your backlog.
 - a.
3. **Find out, and keep the evidence in your repository.** For each unknown, record what you found: the source you used and what you took from it, or a small experiment you ran and what happened. A teammate who did not do the research should be able to follow it. Give each experiment a time limit. Cohn calls it a spike: “a brief experiment to learn about an area of the application.”
 - a.
4. **Test your knowledge by listing the tasks.** Take your top item and list the tasks needed to deliver it. These are a first draft; you will see which belong to which story when you write the stories. Check each task with Wake’s questions: is it specific enough that everyone understands what is involved? Is there a clear point at which you can mark it done? Is it achievable by whoever takes it? Is it relevant to the item? Is it time-boxed, so you know when to ask for help? If you cannot write specific tasks, you do not yet know enough. Return to step 3.
 - a.

Stories – Checkpoint 3

User Story	Acceptance Criteria
As a resident of city X I want to know the fastest route from A to B ,so that I can save time taking my daily commute.	AC1: When I enter location data I want to see all possible routes from my location to the entered location, and see which is fastest.
As a resident in city X I want to find the most cost efficient way to go from A to B, so I can save money.	AC1: When I enter location data I want to see the cost of travel from my current location to the entered one, and see the cheapest mode

	of travel.
As a person traveling the United States I want to know the best mode of transportation from city X to city Y ,so that I can save time and money when traveling long distances.	AC1: When I enter location data for long distance travel I want to see the fastest and cheapest mode of transportation to get to the entered location.

General Notes (Brainstorming)

Information on ordered list

- 1.
2. Aggregate all available information on public (possibly private) transportation i.e. from/to, cost, time from api's or scraping. Transportation styles include planes, bus, train, etc.

We need a specific format for every type of transportation think of large dictionary with from, to, cost, time, etc

3. Create a specific algorithm that will calculate the optimal routes based on some filter (cost, time, etc).
 - a. Possible methods could include graph solutions like **Dijkstra's algorithm** where each city acts as a node and the cost edges are based on the given filter **or** a massive **recursion** problem (open to others). Each change from transportation needs some sort of buffer time to allow user to realistically travel between them (i.e. going from airport to train station)
4. UI could include a list of the output transportation with links to where to buy tickets to each
5. UI could have a map that shows the actual route to be taken along with the individual information about each edge transportation

GPS Coordinates elliptical to filter what stations to include in algorithm search.